

01_Fractal Generative Models 数学过程详解

递归概率分解、树状复杂度与像素级似然的完整推导

本文只处理真正需要推导的部分

本文聚焦递归生成器、块级概率链式分解、树状注意力复杂度、负对数似然和分类器自由引导。全文按“公式从哪里来-每个符号是什么-中间步骤为何成立-维度和复杂度如何核对-论文结论依赖哪些假设”的顺序展开；不复述实验表格，也不使用与论文无关的通用背景凑页数。

目录

1	递归生成器的数学定义	3
2	联合分布的块级链式分解	3
2.1	从逐变量链式法则到块链式法则	3
2.2	第二层递归展开	4
3	负对数似然与 bits-per-dimension	4
4	理想化复杂度推导：为什么短序列递归可能更省	4
4.1	单一长序列注意力	4
4.2	固定分支数的树状短序列	5
4.3	论文像素局部注意力示例的核对	5
5	FractalAR 与随机顺序 FractalMAR 的概率差异	5
6	分类器自由引导的 logits 推导	6
7	参数共享与“分形自相似”的严格含义	6
8	梯度如何穿过递归树	6
9	容易误解的结论	7
10	完整递归树的节点数与计算图	7
11	块级链式法则与像素级链式法则完全等价	7

12 层级注意力复杂度的更精确推导	8
13 递归深度与误差传播	8
14 参数共享下的梯度求和	9
15 随机生成顺序目标是排列期望	9
16 分类器自由引导的概率推导	9
17 一个 4×4 离散图像的完整递归概率展开	11
18 树形链式法则与普通像素链式法则为何等价	11
19 熵的层级分解与 bits-per-dimension	12
20 递归计算复杂度的主定理推导	12
21 参数共享梯度的精确求和	12
22 随机顺序训练目标的无偏性与方差	13
23 分类器自由引导的概率比解释	13
24 递归预测误差如何向叶子传播	14

1 递归生成器的数学定义

论文先把“生成模型”抽象成可递归调用的生成器。第 i 层生成器记为 g_i ，上一层的一个输出 x_i 被映射为下一层的一组输出：

$$\{x_{i+1}^{(1)}, \dots, x_{i+1}^{(k)}\} = g_i(x_i). \quad (1.1)$$

这里 k 是每次递归产生的分支数。若从根节点开始递归 n 层，叶节点数量为

$$N = k^n. \quad (1.2)$$

两边取对数得到

$$n = \log_k N = \frac{\log N}{\log k}. \quad (1.3)$$

这一步解释了论文所谓“线性递归深度产生指数数量输出”：输出维度 N 对层数 n 是指数增长，而层数只随 $\log N$ 增长。

256×256 图像的递归层数

若每个生成器固定产生 $k = 16$ 个子块，而最终需要 $N = 16^4 = 65536$ 个空间位置，则只需 $n = 4$ 层递归。这里“4 层”并不是说网络只有 4 个 Transformer Block，而是指分治树的递归层级。

2 联合分布的块级链式分解

2.1 从逐变量链式法则到块链式法则

对随机变量 $x_{1:N}$ ，概率链式法则为

$$p(x_{1:N}) = \prod_{j=1}^N p(x_j \mid x_{<j}). \quad (2.1)$$

论文不直接用长度为 N 的单一自回归序列，而是把变量分成 k 个连续块。令

$$B_i = (x_{(i-1)k^{n-1}+1}, \dots, x_{ik^{n-1}}), \quad i = 1, \dots, k. \quad (2.2)$$

则原联合分布可严格写成

$$p(x_{1:k^n}) = \prod_{i=1}^k p(B_i \mid B_{<i}). \quad (2.3)$$

这一式只是把逐变量链式法则按块重新分组，并没有做独立性假设。每个条件分布 $p(B_i \mid B_{<i})$ 仍然是高维分布，于是再由下一层生成器递归分解。

2.2 第二层递归展开

把第 i 个块继续划分为 k 个子块 $B_{i,1}, \dots, B_{i,k}$, 则

$$p(B_i | B_{<i}) = \prod_{j=1}^k p(B_{i,j} | B_{<i}, B_{i,<j}). \quad (2.4)$$

代回上一层得到

$$p(x_{1:k^n}) = \prod_{i=1}^k \prod_{j=1}^k p(B_{i,j} | B_{<i}, B_{i,<j}). \quad (2.5)$$

继续递归到叶节点后, 最终仍然恢复完整的联合分布。FractalAR 的关键不是改变概率论, 而是改变条件概率的计算图: 长序列被组织成树状的多个短序列。

3 负对数似然与 bits-per-dimension

训练目标是最大化图像概率, 等价于最小化负对数似然:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\mathbb{E}_{x \sim p_{\text{data}}} \log p_{\theta}(x). \quad (3.1)$$

利用递归链式分解, 单样本损失可以写成所有树节点条件概率的和:

$$-\log p_{\theta}(x) = -\sum_{\ell=0}^{n-1} \sum_{u \in \mathcal{T}_{\ell}} \log p_{\theta}(x_{\text{child}(u)} | x_{\text{context}(u)}). \quad (3.2)$$

其中 $\mathcal{T}_{\ell}$ 是第 ℓ 层节点集合。图像生成常用 bits-per-dimension:

$$\text{bpd}(x) = -\frac{\log_2 p_{\theta}(x)}{D} = -\frac{\log p_{\theta}(x)}{D \log 2}, \quad (3.3)$$

$D = HWC$ 是标量像素通道总数。bpd 的分母必须使用标量维度而不是 patch 数, 否则不同分块方案不能公平比较。

4 理想化复杂度推导: 为什么短序列递归可能更省

4.1 单一长序列注意力

若直接对长度 N 的像素序列做全注意力, 单层注意力矩阵为 $N \times N$, 计算量近似

$$C_{\text{flat}} = O(N^2 d), \quad (4.1)$$

其中 d 为隐藏维度。

4.2 固定分支数的树状短序列

在理想化分析中，假设每个节点只运行长度 k 的注意力，且第 ℓ 层有 k^ℓ 个节点。第 ℓ 层注意力代价为

$$C_\ell = k^\ell \cdot O(k^2 d). \quad (4.2)$$

总成本为几何级数

$$C_{\text{fractal}} = O(k^2 d) \sum_{\ell=0}^{n-1} k^\ell \quad (4.3)$$

$$= O(k^2 d) \frac{k^n - 1}{k - 1} \quad (4.4)$$

$$= O\left(\frac{k^2}{k - 1} N d\right). \quad (4.5)$$

当 k 视为常数时，理想复杂度随 N 近似线性增长。这个结论依赖“每个节点只看固定长度局部序列”和“不同节点可以有效批处理”；真实模型还包含跨层条件 token、不同宽度和重复网络计算，因此不能直接把它当成精确 FLOPs 公式。

4.3 论文像素局部注意力示例的核对

对 256×256 图像，平铺像素注意力矩阵元素数为

$$(256 \cdot 256)^2 = 4\,294\,967\,296. \quad (4.6)$$

若把图像划为 64×64 个 4×4 小块，每块只在 16 个像素上做注意力，则总元素数为

$$(64 \cdot 64) \times 16^2 = 1\,048\,576. \quad (4.7)$$

比值为

$$\frac{4\,294\,967\,296}{1\,048\,576} = 4096. \quad (4.8)$$

这正是局部短序列结构带来的数量级差异，但它只比较注意力矩阵规模，不等于完整模型端到端加速倍数。

5 FractalAR 与随机顺序 FractalMAR 的概率差异

FractalAR 对子块使用固定顺序：

$$p(B_1, \dots, B_k) = \prod_{i=1}^k p(B_i \mid B_{<i}). \quad (5.1)$$

若采用随机顺序 π ，则训练目标对排列取期望：

$$\mathcal{L}_{\text{MAR}} = \mathbb{E}_\pi \left[- \sum_{i=1}^k \log p_\theta(B_{\pi_i} \mid B_{\pi_{<i}}, m_\pi) \right], \quad (5.2)$$

其中 m_π 表示当前可见/遮蔽模式。固定顺序学习一个有向因子分解；随机顺序要求模型在多种条件集合下恢复目标块，训练信号更接近双向条件建模。

6 分类器自由引导的 logits 推导

论文在条件分支和无条件分支分别得到 logits l_c 与 l_u ，再使用

$$l = l_u + \omega(l_c - l_u). \quad (6.1)$$

当 $\omega = 0$ 时完全无条件； $\omega = 1$ 时等于条件 logits； $\omega > 1$ 是沿条件方向外推。若把 logits 写成未归一化对数概率，

$$l_c(y) \approx \log p(y | c) + C_c, \quad l_u(y) \approx \log p(y) + C_u, \quad (6.2)$$

则忽略与类别无关的常数后

$$l(y) \approx (1 - \omega) \log p(y) + \omega \log p(y | c), \quad (6.3)$$

$$\exp l(y) \propto p(y)^{1-\omega} p(y | c)^\omega. \quad (6.4)$$

所以 CFG 实际上重新加权了条件概率与先验概率。它不是把两个 Softmax 概率直接线性平均；若先 Softmax 再组合，结果一般不同。

7 参数共享与“分形自相似”的严格含义

数学分形常要求尺度变换后的严格自相似，而论文中的神经网络分形更接近结构自相似。若所有层共享同一参数 θ ，递归可写为

$$x_{\ell+1} = g_\theta(x_\ell), \quad (7.1)$$

属于真正的迭代函数系统式参数共享。若不同层使用 θ_ℓ ，则

$$x_{\ell+1} = g_{\theta_\ell}(x_\ell), \quad (7.2)$$

保留拓扑结构相似，但不具有参数层面的严格自相似。论文的主要贡献是生成模块的递归组织，而不是证明生成分布具有经典分形维数。

8 梯度如何穿过递归树

设总损失是所有叶节点损失之和：

$$\mathcal{L} = \sum_{v \in \mathcal{L}} \ell_v. \quad (8.1)$$

对某个内部节点表示 h_u ，链式法则给出

$$\frac{\partial \mathcal{L}}{\partial h_u} = \sum_{v \in \text{desc}(u)} \frac{\partial \ell_v}{\partial h_u}. \quad (8.2)$$

因此内部节点汇总其所有后代的梯度。若同一生成器参数被多个节点共享，则

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{u \in T} \frac{\partial \mathcal{L}}{\partial g_{\theta}(h_u)} \frac{\partial g_{\theta}(h_u)}{\partial \theta}. \quad (8.3)$$

共享提高参数效率，但不同尺度的梯度可能冲突；不共享则参数量随层数增加，却允许不同尺度学习不同统计规律。

9 容易误解的结论

三点边界

第一， $n = \log_k N$ 只说明递归深度，不说明运行时间一定是对数级；整棵树仍需生成 N 个叶节点。第二，块链式分解是概率恒等式，但用有限容量网络分别拟合各条件分布会产生近似误差。第三，注意力矩阵元素减少 4096 倍不等于端到端速度提升 4096 倍，因为还有 MLP、投影、递归调度与显存访问。

10 完整递归树的节点数与计算图

设根节点处于第 0 层，每个内部节点产生 k 个子节点，递归深度为 n 。第 ℓ 层节点数为

$$N_{\ell} = k^{\ell}, \quad (10.1)$$

全部内部节点数为

$$N_{\text{int}} = \sum_{\ell=0}^{n-1} k^{\ell} = \frac{k^n - 1}{k - 1}. \quad (10.2)$$

叶节点数为 $k^n = N$ ，总节点数

$$N_{\text{all}} = \frac{k^{n+1} - 1}{k - 1}. \quad (10.3)$$

若每个内部节点都调用同一参数共享生成器 g_{θ} ，参数量不会乘以 N_{int} ，但前向调用次数会乘以该数量。这一区分是理解“参数效率”和“计算效率”的关键。

11 块级链式法则与像素级链式法则完全等价

设一维序列被划分成 k 个块 $B_1, \dots, B_k$ 。联合概率的链式法则给出

$$p(B_{1:k}) = p(B_1) \prod_{i=2}^k p(B_i \mid B_{<i}). \quad (11.1)$$

再对每个块内部展开

$$p(B_i \mid B_{<i}) = \prod_{j=1}^{|B_i|} p(x_{i,j} \mid B_{<i}, x_{i,<j}). \quad (11.2)$$

把两式相乘，条件集合恰好覆盖所有在全局顺序中位于当前变量之前的变量，因此

$$\prod_i \prod_j p(x_{i,j} \mid B_{<i}, x_{i,<j}) = \prod_{m=1}^N p(x_m \mid x_{<m}). \quad (11.3)$$

所以分块没有改变概率模型的合法性；它改变的是条件概率由哪种计算图实现。

12 层级注意力复杂度的更精确推导

在第 ℓ 层有 k^ℓ 个父节点，每个父节点对 k 个子块做局部注意力。若隐藏维度为 d ，单个局部注意力的主项为

$$C_{\text{node}} \approx 4kd^2 + 2k^2d, \quad (12.1)$$

其中 $4kd^2$ 来自 Q, K, V, O 线性映射， $2k^2d$ 来自 $QK^\top$ 与概率加权。全树成本

$$C_{\text{tree}} \approx \sum_{\ell=0}^{n-1} k^\ell (4kd^2 + 2k^2d) \quad (12.2)$$

$$= \frac{k^n - 1}{k - 1} (4kd^2 + 2k^2d). \quad (12.3)$$

用 $N = k^n$ 表示

$$C_{\text{tree}} = O(Nkd^2 + Nk^2d). \quad (12.4)$$

当 k 固定时对 N 近似线性；但如果为了减少深度而让 k 随 N 增长， k^2 项会重新变贵，因此不能只引用 $O(N)$ 而忽略分支数。

13 递归深度与误差传播

设每一层生成映射对输入的 Jacobian 谱范数满足

$$\|J_{g_\theta}(h)\|_2 \leq L. \quad (13.1)$$

根节点扰动 δh_0 传播到深度 n 的某一叶节点满足

$$\|\delta h_n\| \leq \prod_{\ell=0}^{n-1} \|J_{g_\theta}(h_\ell)\|_2 \|\delta h_0\| \leq L^n \|\delta h_0\|. \quad (13.2)$$

若 $L > 1$ ，误差可能指数放大；若 $L < 1$ ，深层特征可能收缩。由于 $n = \log_k N$ ，上界还可写成

$$L^n = N^{\log_k L}. \quad (13.3)$$

这说明递归深度虽为对数，但稳定性仍取决于共享生成器的局部 Lipschitz 常数。

14 参数共享下的梯度求和

同一参数 θ 在所有内部节点复用。总损失为叶节点损失之和

$$\mathcal{L}(\theta) = \sum_{v \in \mathcal{L}} \ell_v. \quad (14.1)$$

链式法则得到

$$\nabla_{\theta} \mathcal{L} = \sum_{u \in \mathcal{I}} \left(\frac{\partial h_u}{\partial \theta} \right)^{\top} \nabla_{h_u} \mathcal{L}, \quad (14.2)$$

其中 $\mathcal{I}$ 为内部节点集合。参数共享的效果是来自不同空间位置、不同尺度的梯度在同一 θ 上累加。若不同层级任务冲突，梯度内积

$$\langle g_{\ell_1}, g_{\ell_2} \rangle < 0 \quad (14.3)$$

会导致更新相互抵消；这也是共享带来正则化同时可能限制容量的原因。

15 随机生成顺序目标是排列期望

若训练时从分布 $q(\pi)$ 抽取块排列 π ，目标为

$$\mathcal{L}(\theta) = \mathbb{E}_{\pi \sim q} \left[- \sum_i \log p_{\theta}(B_{\pi_i} \mid B_{\pi_{<i}}, \pi) \right]. \quad (15.1)$$

单次排列给出的随机梯度

$$\hat{g}(\pi) = \nabla_{\theta} \mathcal{L}_{\pi} \quad (15.2)$$

满足

$$\mathbb{E}_{\pi} \hat{g}(\pi) = \nabla_{\theta} \mathbb{E}_{\pi} \mathcal{L}_{\pi}, \quad (15.3)$$

只要可交换微分与期望。因此随机顺序训练是无偏 Monte Carlo 优化；代价是梯度方差

$$\text{Var}_{\pi}[\hat{g}] = \mathbb{E} \|\hat{g} - \mathbb{E} \hat{g}\|^2 \quad (15.4)$$

可能较大，需要更多排列或控制变量降低。

16 分类器自由引导的概率推导

条件与无条件 score 分别为

$$s_c(x) = \nabla_x \log p(x \mid c), \quad s_u(x) = \nabla_x \log p(x). \quad (16.1)$$

引导 score

$$s_w = (1 + w)s_c - ws_u \quad (16.2)$$

可写为某个未归一化密度的梯度：

$$s_w = \nabla_x [(1 + w) \log p(x | c) - w \log p(x)] \quad (16.3)$$

$$= \nabla_x \log \frac{p(x | c)^{1+w}}{p(x)^w}. \quad (16.4)$$

因此 CFG 相当于从

$$\tilde{p}_w(x | c) \propto p(x | c)^{1+w} p(x)^{-w} \quad (16.5)$$

采样，强化条件相关方向。 w 太大时该分布更尖锐，覆盖率下降并可能产生过饱和。

17 一个 4×4 离散图像的完整递归概率展开

为了把“递归生成”从口号变成可计算对象，考虑每个像素只能取 K 个离散值的 4×4 图像 X 。先把图像分成四个 2×2 子块

$$X = (B_1, B_2, B_3, B_4), \quad B_i \in \{1, \dots, K\}^{2 \times 2}. \quad (17.1)$$

顶层模型先生成四个块的摘要变量 $C_1, \dots, C_4$ 。若采用固定顺序，则联合分布写成

$$p(C_1, C_2, C_3, C_4) = p(C_1) \prod_{j=2}^4 p(C_j | C_{<j}). \quad (17.2)$$

每个块内部再递归分解为四个像素。以 $B_2 = (x_{21}, x_{22}, x_{23}, x_{24})$ 为例，

$$p(B_2 | C_2) = p(x_{21} | C_2) p(x_{22} | x_{21}, C_2) p(x_{23} | x_{21:22}, C_2) p(x_{24} | x_{21:23}, C_2). \quad (17.3)$$

因此完整图像概率并不是“多个独立小模型概率的乘积”，而是

$$p(X) = p(C_{1:4}) \prod_{j=1}^4 p(B_j | C_j, C_j), \quad (17.4)$$

其中 C_j 表示顶层已经生成的上下文。取对数后得到

$$-\log p(X) = -\log p(C_{1:4}) - \sum_{j=1}^4 \sum_{r=1}^4 \log p(x_{jr} | x_{j,<r}, C_j, C_j). \quad (17.5)$$

这一步说明：递归结构只改变了条件概率的组织方式，并没有改变最大似然的基本原则。

18 树形链式法则与普通像素链式法则为何等价

把所有叶子像素按递归遍历顺序记为 $x_{\pi(1)}, \dots, x_{\pi(N)}$ 。概率链式法则对任何排列 π 都成立：

$$p(x_{1:N}) = \prod_{i=1}^N p(x_{\pi(i)} | x_{\pi(<i)}). \quad (18.1)$$

树中的内部节点本质上是确定性摘要 $c_v = g_v(x_{\text{desc}(v)})$ 或训练时引入的辅助变量。若 c_v 是确定函数，则

$$p(x, c) = p(x) \prod_v \mathbf{1}\{c_v = g_v(x)\}, \quad (18.2)$$

边缘化内部节点得到

$$\sum_c p(x, c) = p(x). \quad (18.3)$$

所以“分形”并没有凭空增加概率质量；它只是把长序列分成多层较短条件序列。若内部节点是随机变量，则必须显式积分或求和：

$$p(x) = \sum_c p(c) p(x | c), \quad (18.4)$$

训练时对应的负对数似然不能随意省略 c 的边缘化。

19 熵的层级分解与 bits-per-dimension

对离散图像，平均负对数似然为

$$\mathcal{L}_{\text{NLL}} = -\mathbb{E}_{X \sim p_{\text{data}}} \log_2 p_{\theta}(X). \quad (19.1)$$

若图像共有 $D = HWC$ 个标量，则

$$\text{bpd} = \frac{\mathcal{L}_{\text{NLL}}}{D}. \quad (19.2)$$

对层级变量 (C, X) ，熵链式法则给出

$$H(X, C) = H(C) + H(X | C). \quad (19.3)$$

若 C 是 X 的确定函数，则 $H(C | X) = 0$ ，于是

$$H(X) = H(C) + H(X | C). \quad (19.4)$$

这正是顶层模型和局部递归模型分别承担的最小理论码长。若顶层摘要压缩得过强， $H(X | C)$ 会增大；若摘要过细， $H(C)$ 会增大。两者之间存在信息瓶颈式权衡。

20 递归计算复杂度的主定理推导

设一个长度为 n 的区域被分成 b 个等规模子区域，每个节点的局部 Transformer 处理 b 个子块摘要，局部开销记为 $g(b, d)$ 。若 b 固定，则递推式为

$$T(n) = bT(n/b) + g(b, d). \quad (20.1)$$

递归深度为 $L = \log_b n$ ，第 ℓ 层有 b^ℓ 个节点，因此

$$T(n) = \sum_{\ell=0}^{L-1} b^\ell g(b, d) = g(b, d) \frac{b^L - 1}{b - 1} = O(n g(b, d)). \quad (20.2)$$

若每个节点只对固定数目的子块做注意力， $g(b, d) = O(b^2 d)$ 是常数阶的，故总复杂度对叶子数近似线性。相比之下，直接对 n 个像素做全注意力是

$$T_{\text{full}}(n) = O(n^2 d). \quad (20.3)$$

但该线性结论依赖两个条件：分支数固定、跨兄弟子树的全局信息只通过摘要传递。若每层还对全部节点做全局注意力，第 ℓ 层开销会成为 $O(b^{2\ell} d)$ ，总和重新回到二次量级。

21 参数共享梯度的精确求和

同一个递归模块 f_{θ} 在树上被调用 M 次，节点 v 的输出为

$$h_v = f_{\theta}(s_v). \quad (21.1)$$

总损失 $\mathcal{L} = \sum_{u \in \mathcal{L}} \ell_u$ 对共享参数的梯度为

$$\nabla_{\theta} \mathcal{L} = \sum_{v=1}^M \left(\frac{\partial h_v}{\partial \theta} \right)^{\top} \nabla_{h_v} \mathcal{L}. \quad (21.2)$$

这里每个调用点的局部 Jacobian 都不同，因为输入状态 s_v 不同。参数共享减少了存储参数量，却不会把计算图中的 M 次调用压缩成一次；反向传播仍需累计所有节点贡献。

进一步，若一条根到叶路径长度为 L ，状态 Jacobian 连乘为

$$\frac{\partial h_L}{\partial h_0} = \prod_{\ell=1}^L J_{\ell}, \quad J_{\ell} = \frac{\partial h_{\ell}}{\partial h_{\ell-1}}. \quad (21.3)$$

于是

$$\left\| \frac{\partial h_L}{\partial h_0} \right\|_2 \leq \prod_{\ell=1}^L \|J_{\ell}\|_2. \quad (21.4)$$

若平均谱范数大于 1，梯度可能爆炸；小于 1，梯度可能衰减。残差连接把 J_{ℓ} 改成 $I + J_{F,\ell}$ ，可以保留恒等梯度通道。

22 随机顺序训练目标的无偏性与方差

FractalMAR 类方法对生成排列 π 取随机样本。完整目标是

$$\mathcal{L}(\theta) = \mathbb{E}_{\pi \sim q(\pi)} \mathbb{E}_{X \sim p_{\text{data}}} \left[- \sum_{i=1}^N \log p_{\theta}(x_{\pi(i)} \mid x_{\pi(<i)}, \pi) \right]. \quad (22.1)$$

每次只采一个排列得到随机梯度 $g(\pi)$ ，若满足可交换微分与期望的条件，

$$\mathbb{E}_{\pi}[g(\pi)] = \nabla_{\theta} \mathcal{L}(\theta), \quad (22.2)$$

因此是一阶无偏估计。但其方差为

$$\text{Var}[g(\pi)] = \mathbb{E} \|g(\pi) - \mathbb{E} g(\pi)\|_2^2, \quad (22.3)$$

不同排列导致上下文差异越大，方差越高。一个直接的降方差办法是在同一图像上采样多个排列并平均：

$$\bar{g}_K = \frac{1}{K} \sum_{k=1}^K g(\pi_k), \quad \text{Var}(\bar{g}_K) = \frac{1}{K} \text{Var}(g), \quad (22.4)$$

代价是计算量约增大 K 倍。

23 分类器自由引导的概率比解释

设条件与无条件 logits 对应分布 $p_c(x)$ 与 $p_u(x)$ 。引导 logits

$$\ell_{\text{cfg}} = (1 + w)\ell_c - w\ell_u \quad (23.1)$$

经过 softmax 后满足

$$p_{\text{cfg}}(x) \propto \exp((1+w)\ell_c(x) - w\ell_u(x)) \propto \frac{p_c(x)^{1+w}}{p_u(x)^w}. \quad (23.2)$$

因此 $w > 0$ 实际上放大条件分布相对于无条件分布的似然比。它提高条件一致性，但也会使分布变尖，降低多样性。若 $p_u(x)$ 在某些事件上极小，概率比会非常大，数值实现中应在 logits 域计算而不是直接做概率除法。

24 递归预测误差如何向叶子传播

设父节点摘要误差为 e_ℓ ，下一层局部生成器对父状态的 Lipschitz 常数为 L_ℓ ，自身近似误差为 δ_ℓ ，则

$$e_{\ell+1} \leq L_\ell e_\ell + \delta_\ell. \quad (24.1)$$

递推展开得到

$$e_L \leq \left(\prod_{j=0}^{L-1} L_j \right) e_0 + \sum_{k=0}^{L-1} \left(\prod_{j=k+1}^{L-1} L_j \right) \delta_k. \quad (24.2)$$

这说明高层摘要的错误会影响整棵子树；越靠近根部的误差乘上的 Lipschitz 因子越多。训练时只看最终像素损失虽然能端到端纠错，但也会造成高层信用分配困难，因此分层辅助损失具有明确的数值稳定意义。